

STOR 538 Playoff Project: NBA Predictions

Group 10

Levin Wu, Lena Osborne, Maya Lennon, Kyran Lakhia

Table Of Contents

Data Information	3
Methodology for Spread.....	11
Methodology for Total.....	15
Methodology for OREB.....	22

Data Information

Base Game Dataset

We started by working to create a master dataset to base our predictive models on. Our group utilized multiple NBA game-level and team-level datasets found in Kaggle. The datasets are found in NBA Database (1947 - Present) and NBA Stats (1947 - Stats). These datasets include Games.csv for game outcomes and structure, Opponent Stats Per Game.csv for season-level defensive context, Team Stats Per 100 Poss.csv for game-level box-score calculations, and Team Stats Per Game.csv and Team Statistics.csv for rebound opportunity and pace-adjusted rebounding strength. The datasets found under NBA Database (Games.csv & Team Statistics.csv) in particular were set up such that they update nightly with game-based information from the box scores. We combined these dataframes and engineered additional variables designed to improve the prediction of three response variables: total points, point spread, and total offensive rebounds. The master CSV file was created such that another person could follow the same steps and build their own models from it.

We began with historical NBA game results datasets found Games.csv. This file provided the base structure containing variables such as game identifier, game date, home team, etc. From this, we created an initial master set with the variables game_id, game_date, home_team, away_team, home_pts, and away_pts. We then engineered two variables: spread and total. They were defined as:

- $spread = home_pts - away_pts$
- $total = home_pts + away_pts$

Additionally, the season variable was engineered based on the game_date variable. For our purposes, games in October, November, and December were assigned to the following calendar

year. For example, a game on November 11th, 2025 would be listed as 2026 for the season variable.

Opponent Defensive Statistics

Next, we incorporated team defensive statistics from Opponent Stats Per Game.csv. This included season-level data describing how teams perform defensively against their opponents.

The following variables were extracted:

opp_fg_per_game	Average number of field goals made by opponents per game against the team. Measures how many made shots the team allows defensively.
opp_fga_per_game	Average number of field goal attempts taken by opponents per game against the team. Indicates how many shot opportunities the defense allows.
opp_x3p_per_game	Average number of three-point field goals made by opponents per game. Measures how many made three-point shots the team allows.
opp_x3pa_per_game	Average number of three-point field goal attempts taken by opponents per game. Reflects opponent three-point shooting volume against the defense.
opp_orb_per_game	Average number of offensive rebounds secured by opponents per game. Measures how often opponents recover their own missed shots against the team.
opp_drb_per_game	Average number of defensive rebounds secured by opponents per game. Indicates how often opponents finish possessions with defensive rebounds.
opp_trb_per_game	Average total rebounds (offensive + defensive) secured by opponents per game. Provides an overall measure of opponent rebounding success.
opp_ast_per_game	Average number of assists recorded by opponents per game. Indicates how frequently opponents generate assisted scoring opportunities.
opp_tov_per_game	Average number of turnovers committed by opponents per game. Reflects how often the team forces turnovers defensively.
opp_pts_per_game	Average number of points scored by opponents per game against the team. Measures the overall scoring allowed by the defense.

The team names in the two datasets were stored in different formats. To fix this, we created a team lookup table matching shortened team names to their full versions. This allowed us to join it with the master set. The opponent statistics set was joined twice: once for the home team

(home_...) and once for the away team (away_...). Once this was done, the following variables were engineered to capture defensive differences between the two teams:

- $opp_pts_allowed_diff = home_opp_pts_per_game - away_opp_pts_per_game$
- $opp_orb_allowed_diff = home_opp_orb_per_game - away_opp_orb_per_game$
- $opp_trb_allowed_diff = home_opp_trb_per_game - away_opp_trb_per_game$
- $opp_ast_allowed_diff = home_opp_ast_per_game - away_opp_ast_per_game$
- $opp_tov_forced_diff = home_opp_tov_per_game - away_opp_tov_per_game$

These capture differences in defensive tendencies that may influence scoring, possessions, and rebounding

Box Score Efficiency Metrics

We then incorporated game-level box scores stats from Team Statistics.csv which contained the following raw stats for each game:

fieldGoalsMade, fieldGoalsAttempted, threePointersMade, threePointersAttempted, freeThrowsMade, freeThrowsAttempted, reboundsOffensive, reboundsDefensive, turnovers, teamScore.

From these variables, we engineered several advanced efficiency metrics commonly used in basketball analytics:

Effective FG Percentage	$efg_pct = (FGM + 0.5 \times 3PM) / FGA$
Free Throw Rate	$ft_rate = FTA / FGA$
Turnover Percentage	$tov_pct = TOV / (FGA + 0.44 \times FTA + TOV)$
Possession Estimate	$possessions = FGA - OREB + TOV + 0.44 \times FTA$

Using *possessions* we calculated the following efficiency stats. They summarize team efficiency on a per-possession basis:

Offensive Rating	$off_rtg = 100 \times \frac{points}{possessions}$
Defensive Rating	$def_rtg = 100 \times opponent_points / opponent_possessions$
Net Rating	$net_rtg = off_rtg - def_rtg$
Pace	$pace = (possessions + opponent_possessions) / 2$
OREB Percentage	$orb_pct = OREB / (OREB + opponent_DREB)$

Rolling Performance Metrics

To capture a team's recent performance, we calculated five-game rolling averages for several variables. These variables utilized lagged observations using only past data. For each team, the following were created:

$$last5_off_rtg = mean(off_rtg_{t-1} \dots off_rtg_{t-5})$$

$$last5_def_rtg = mean(def_rtg_{t-1} \dots def_rtg_{t-5})$$

$$last5_net_rtg = mean(net_rtg_{t-1} \dots net_rtg_{t-5})$$

$$last5_pace = mean(pace_{t-1} \dots pace_{t-5})$$

After computing these team-level values, we created several different values to measure the relative strength between two teams. These included `off_rtg_diff`, `def_rtg_diff`, `net_rtg_diff`, `pace_diff`, `efg_pct_diff`, `tov_pct_diff`, `orb_pct_diff`, and `ft_rate_diff`. These rolling statistics capture recent team performance trends, which are often more predictive of upcoming games than full-season averages.

OREB Outcome Variable

The offensive rebound variable was created via the same Team Statistics.csv dataset. Home and away OREB was extracted separately and merged into the master dataset. The variable was defined as $oreb = home_oreb + away_oreb$.

Rolling Game Outcome Variables

We then created additional rolling, average-based features created using recent game outcomes. For each team we found $last5_pts = mean(points_{t-1} \dots points_{t-5})$,

$last5_pts_allowed = mean(points_allowed_{t-1} \dots points_allowed_{t-5})$, and

$last5_oreb = mean(oreb_{t-1} \dots oreb_{t-5})$.

$last5_pts_diff$, $last5_pts_allowed_diff$, and $last5_oreb_diff$ were created from these variables as well to capture recent offensive, defensive, and rebounding performance.

Schedule Variables

The following variables were created to capture rest and fatigue caused by frequency of games:

- $rest_days = date - previous_game_date$
- A binary *back_to_back* indicator (1 if $rest_days = 1$, 0 if otherwise)
- $rest_diff = home_rest_days - away_rest_days$

These variables capture fatigue and rest advantages, which can affect scoring and rebounding performance.

Rebounding Opportunity Variables

To aid the models for OREB, we incorporated the `Team Stats Per Game.csv` datasets, which contain season averages. We engineered several variables to reflect the number of potential rebound opportunities:

- $missed_fg = FGA - FGM$
- Three pt attempt rate: $3PA\ rate = 3PA/FGA$
- $expectedd_missed_fg = home_missed_fg + away_missed_fg$

We then utilized `TeamStatsPer100Possessions.csv`, which provides pace-adjusted rebounding metrics including orb_per100 , drb_per100 , and trb_per100 . From these, the following matchup edge variables were created to measure how offensive rebounding strength compares to defensive rebounding ability:

- $home_rebound_edge = home_orb_{100} - away_drb_{100}$
- $away_rebound_edge = away_orb_{100} - home_drb_{100}$
- $rebound_matchup_edge = home_rebound_edge - away_rebound_edge$

Additional variables were engineered to further estimate rebounding opportunity:

- $expected_oreb_pressure = (home_last5_missed_fg + away_last5_missed_fg) + \frac{home_orb_{100} + away_orb_{100}}{200}$
- $home_oreb_matchup = home_orb_{100} \times away_last5_oreb_allowed$
- $away_oreb_matchup = away_orb_{100} \times home_last5_oreb_allowed$
- $pace_rebound_pressure = \frac{home_last5_pace + away_last5_pac}{2} \times home_last5_missed_fg + away_last5_missed_fg$

Scoring Environment Variables

To aid our total points predictive models, nine variables were created as well. These first five are rolling:

- $shooting_env5 = mean(eFG\%_{t-1} \dots eFG\%_{t-5})$
- $pace_env5 = mean(pace_{t-1} \dots pace_{t-5})$
- $free_throw_env5 = mean(ft_rate_{t-1} \dots ft_rate_{t-5})$
- $tov_env5 = mean(tov_pct_{t-1} \dots tov_pct_{t-5})$
- $blend_total_proj5 = \frac{shooting_env5 + pace_env5 + free_throw_env5 + tov_env5}{4}$

Finally, these last four interaction variables were engineered:

- $pace_efficiency = pace_pressure5 \times shooting_env5$
- $extra_possession_pressure = expected_oreb_pressure + rebound_matchup_edge$
- $expected_possessions = pace_pressure5 + pace_rebound_pressure$
- $scoring_efficiency = \frac{shooting_env5 + free_throw_env5}{2}$

These features summarize game pace, shooting efficiency, and possession stats, which are key determinants of total points scored.

Final Master Dataset Construction

The dataset was then restricted to games played during the modern era (2010 onward) to keep it as close to reality as possible. Additionally, measures were taken to remove as many preseason games as possible. For example, any game that was not an NBA team vs. another NBA team was removed. Additionally, because there wasn't a specific variable describing if a game was preseason, we filtered such that games before October 18th were not included. This is because games before that date are typically preseason matchups. Missing schedule variables

were replaced with 0 where appropriate and games without sufficient historical data for the creation of the rolling variables *home_last5_pts* and *away_last5_pts* (they were marked as NA) were removed. The final modeling dataset was saved as `FINAL_MODEL_READY.csv`. It included all predictor and outcome variables required for modeling. It was then pushed to teammates for analysis. Because `Games.csv` and `Team Statistics.csv` were maintained as updated historical files, the same pipeline could be rerun on the most recent available data to generate current-season predictors without needing a separate live-data source.

Methodology for Spread

For our approach, we based our model on a walk-forward validation approach, where we first took the first two months of the 2025-2026 season as a testing set to train a linear model and then looped the training by incrementing the test set by 1 day until we reached March 13th, 2026. For our linear model, we first had to create multiple new rolling spread variables that calculated the overall spread, away spread, home spread, and variations of those 3 variables, only including winning games. We also used multiple other variables based on the average of the last 5/10 games, like last5_off_rtg_diff, and lagged them before lagging them to ensure that all predictions that were made with these variables would be information we would have for predicting future games. To reduce variables further, we took the difference between each team's respective rolling variables.

After running this linear model, we thinned down the variables we used to 4: "overall_spread_roll_diff", "win_spread_roll_diff", "home_spread_roll_diff", "last5_net_rtg_diff", as those had the best p-values, and by removing the extra variables, we reduced the noise in our linear model. To compare the linear model, we created a baseline model that just used overall_spread_roll_diff to predict the spread. From this comparison the linear model with an RMSE of 15.33 and an MAE of 12.306 outperformed the baseline model by 0.423 RMSE and 0.128 MAE in the 2026 season.

Final Linear Model Equation:

$$\text{Spread}_t = \beta_0 + \beta_1 \cdot \text{overall_spread_roll_diff}_t + \beta_2 \cdot \text{win_spread_roll_diff}_t + \beta_3 \cdot \text{home_spread_roll_diff}_t + \beta_4 \cdot \text{last5_net_rtg_diff}_t$$

- overall_spread_roll_diff_t = home overall rolling spread – away overall rolling
- win_spread_roll_diff_t = home winning rolling spread – away winning rolling

- $\text{home_spread_roll_diff}_t = \text{home home rolling spread} - \text{away away rolling}$
- $\text{last5_net_rtg_diff}_t = \text{difference in net rating over the last 5 games}$

Baseline Model Equation:

$$\text{Spread}_t = \beta_0 + \beta_1 \cdot \text{overall_spread_roll_diff}_t -$$

Our next step was to compare to create a new adjusted model that used the linear model to incorporate rolling averages for home and away spreads. For these adjusted models, if the linear model predicted a home win, we considered the home team's overall rolling spread, home spread, and winning spreads, and similarly for the away team if an away win was predicted. These adjusted predictions were only applied if the sign of the rolling average matched the direction of the linear model prediction. The logic behind this decision was that when each team won, they would likely win around their average spread.

Adjusted 1: Uses the predicted winner's overall rolling spread if it aligns with the linear model sign:

$$\hat{y}_t^{\text{adj1}} = \begin{cases} \text{home_overall_spread_roll}_t & \text{if home predicted to win and sign matches} \\ -\text{away_overall_spread_roll}_t & \text{if away predicted to win and sign matches} \\ \hat{y}_t^{\text{lin}} & \text{otherwise} \end{cases}$$

Adjusted 2: Uses winning spreads only:

$$\hat{y}_t^{\text{adj2}} = \begin{cases} \text{home_win_spread_roll}_t & \text{if home predicted to win and sign matches} \\ -\text{away_win_spread_roll}_t & \text{if away predicted to win and sign matches} \\ \hat{y}_t^{\text{lin}} & \text{otherwise} \end{cases}$$

Adjusted 3: Uses home/away specific rolling spreads:

$$\hat{y}_t^{\text{adj3}} = \begin{cases} \text{home_home_spread_roll}_t & \text{if home predicted to win and sign matches} \\ -\text{away_away_spread_roll}_t & \text{if away predicted to win and sign matches} \\ \hat{y}_t^{\text{lin}} & \text{otherwise} \end{cases}$$

Adjusted 4: Uses home/away winning spreads:

$$\hat{y}_t^{\text{adj4}} = \begin{cases} \text{home_home_win_spread_roll}_t & \text{if home predicted to win and sign matches} \\ -\text{away_away_win_spread_roll}_t & \text{if away predicted to win and sign matches} \\ \hat{y}_t^{\text{lin}} & \text{otherwise} \end{cases}$$

However, these 4 models all underperformed compared to the linear model. With Adjusted 1 and 3 underperforming by 0.3-0.5 RMSE, and the other 2 underperforming by around 3. The significant underperformance of both 2 and 4 is likely due to the high variance each spread could have, due to some teams not winning many games. To refine the adjustments, we implemented a thresholding approach for each adjusted model. Using a grid search, we tested different threshold values to determine when to rely on the rolling averages versus the linear model directly. After implementing the thresholds, we validated the method by applying it to the 2025 season while pretending we did not know any game results past March 13th. This walk-forward testing allowed us to evaluate which threshold gave the most accurate predictions. The analysis showed that the threshold 1 model with a value of 2 performed best. This low threshold does suggest that the linear model is slightly worse at predicting close games, resulting in the low threshold of 2.

Threshold Model Equation for the Threshold 1 model (Other models are similar, just replace it with their respective adjusted model):

$$\hat{y}_t^{\text{thr1}} = \begin{cases} \hat{y}_t^{\text{adj1}} & \text{if } |\hat{y}_t^{\text{lin}}| \leq T \\ \hat{y}_t^{\text{lin}} & \text{otherwise} \end{cases}$$

With $T = 2$

This table summarises the average RMSE and MAE when validating our model with the 2025 season.

Model	RMSE	MAE
Linear	15.37109	12.30286
Baseline	15.56392	12.42648
Adjusted 1	15.81938	12.66261
Adjusted 2	18.65761	12.96284
Adjusted 3	16.28216	12.96284
Adjusted 4	19.48240	15.72286
Threshold 1	15.79527	12.64503
Threshold 2	18.53175	14.97755
Threshold 3	16.17898	12.87511
Threshold 4	19.25902	15.51694

From there, we finished our predictions by using the Threshold 1 model with a threshold of 2 to complete our predictions using the 2026 season data. Future improvements could include adding more features like player stats or injuries, using possible better performing non-linear models to predict wins and act as a fallback, implementing dynamic thresholds (changing results based on close vs average vs blowout games), and incorporating game context such as travel or back-to-back games, which could further improve predictive accuracy.

Methodology for Total

Before we created our models, we examined the distribution of the response variable, total combined points scored per game. We found that NBA game totals range from 170 to 290 points and have a slight right skew (skewness ≈ 0.167). To account for this, we fit parallel versions of each model on both the raw scale and the log-transformed scale, back-transforming log-scale predictions via $\exp()$ before computing MAE, and reported whichever version produced the lower test-set error.

Additionally, because NBA statistics are computed as rolling windows, a random train/test split would allow future game outcomes to contaminate the predictors of past games. This is a form of data leakage that would produce misleadingly optimistic error estimates. We therefore enforced a strict temporal split, so that all games before February 20, 2026 form the training set (19,660 games spanning October 2009 through February 2026), and all games on or after that date form the test set (160 games through March 12, 2026). This helps to directly mirror the real-world prediction task of forecasting March and April 2026 games. Cross-validation within the training set was used exclusively to tune hyperparameters and we made sure to never use it to evaluate final test performance. We make this abundantly clear because using the test set for any tuning decision could cause a data leakage and ultimately inflate reported accuracy. Because our data is time-ordered, standard k-fold cross-validation is inappropriate; randomly assigned folds would allow a model to train on future observations and validate on past ones, which is impossible in the real-world. To preserve temporal ordering throughout, we applied two CV strategies depending on the computational cost of each model.

For most models, we used caret's timeslice method, a time-series-aware sliding window approach. We chose this because it begins with an initial training window covering 60% of the training observations and a validation horizon of 8% of observations (~1,573 games). At each step, the window slides forward in time, so validation observations always occur after all training observations used in that fold. This ensures the cross-validation setup is structurally identical to the final temporal train/test split, making CV-estimated errors a reliable proxy for out-of-sample performance on future games. Although for the stacking ensemble, we fell back to standard 5-fold CV, and just had to accept that this could cause a small optimism bias in the CV error estimates. In all cases, hyperparameters were selected by the combination that minimized cross-validated MAE, and the final model was refitted on the full training set using those optimal settings before being evaluated once on the held-out test set.

The baseline model we created predicts the training-set mean total (213.1 points) for every game, regardless of matchup. This base model returned a test MAE of 20.9 points and serves as the floor that every more sophisticated model must beat. Any model that fails to outperform the training mean offers no practical predictive value, but luckily, no model performed worse than the baseline. We first fit a full OLS regression with all 55 predictors. The model was overall okay, but it did have severe multicollinearity: several variance inflation factors (VIFs) exceeded 100, particularly among the various rolling differential pairs that are nearly redundant (e.g., `last5_off_rtg_diff` and `last5_net_rtg_diff`). To address this, we fit a pruned OLS model retaining 15 predictors, in order to minimize redundancy while also maintaining coverage of each conceptual predictor cluster, which turned out to be better, as expected. We also fit an OLS model on $\log(\text{total})$, back-transforming predictions with $\exp()$; the log model improved the MAE, as expected, because it is consistent with the mild right skewness in the

response. Because OLS is unstable under multicollinearity, we applied L1 (Lasso) and L2 (Ridge) regularization using the `glmnet` package. For each, the penalty parameter λ was selected by 5-fold CV in order to minimize MAE. Ridge keeps all the predictors but shrinks correlated ones toward each other, while Lasso performs variable selection by driving some coefficients to exactly zero. Both were competitive with pruned OLS. The Lasso automatically zeroed out approximately 20 of the 55 predictors, effectively replicating the manual pruning but with a data-driven mechanism instead of us doing so visually.

Moving on to more advanced models, we decided to create a random forest that builds 500 decision trees on bootstrap samples of the training data and averages their predictions. At each split, only a random subset of `mtry` predictors are considered as candidates, which decorrelates the trees and reduces variance. We tuned `mtry` via 5-fold CV over three values near the regression default of $\text{floor}(\sqrt{p})$: $\text{floor}(\sqrt{55}) \approx 7$, $\text{floor}(55/4) = 13$, and $\text{floor}(55/3) = 18$. The optimal `mtry` was selected by lowest CV MAE and used to train the final forest. Random forest produced a considerable improvement over OLS, achieving a 13.5-point MAE. Variable importance plots (measured by %IncMSE or the percent increase in MSE when a variable is permuted) consistently identified `blend_total_proj5`, `pace_pressure5`, and `shooting_env5` as the most informative predictors. We also decided to do a gradient boosted tree. It builds sequentially, with each new tree fitting the residuals of the current ensemble. Unlike random forest's variance-reduction strategy, boosting mainly reduces bias through using iterative correction. The GBM was our most complex model and required the most careful tuning. We searched a grid of 108 hyperparameter combinations using 5-fold CV:

Hyperparameter	Values Searched	Description
n.trees	1000, 2000	Number of trees(boosting iterations)
interaction.depth	2, 3	Maximum tree depth
shrinkage	0.01, 0.03	Learning rate
n.minobsinnode	10, 20	Minimum observations per terminal node
bag.fraction	0.75	Fraction of training data sampled per tree
distribution	guassian	Loss function matching to regression on raw scale

The optimal hyperparameters were selected by lowest cross-validated MAE and used to train the final GBM on all training data. We also trained a parallel GBM on log(total) with identical hyperparameters, back-transforming predictions via $\exp()$, and selected whichever version, raw or log scale, achieved lower test MAE. The GBM was our best individual model, achieving approximately 16.24 points MAE on the held-out part 2025-26 season. The GBM variable importance (measured by the relative influence, which is the fraction of total reduction in squared error attributable to each predictor) again highlighted `pace_efficiency`, `blend_total_proj5`, and `pace_pressure5` as dominant predictors, followed by `off_rtg_diff` and `shooting_env5`. Variables with almost zero relative influence included `home_back_to_back`, `away_back_to_back` in isolation (though `rest_diff` was more useful), and the missed-rebound differential columns (`last5_missed_3pt_diff`, `x3pa_rate_per100_diff`), which could be noisy proxies for information already captured in the composite features. Next, a single-hidden-layer feedforward neural network was fitted using the `nnet` package. Inputs were first scaled to $[0, 1]$ using min-max normalization (fitted on training data only, applied to both sets) to ensure stable gradient descent. We tuned the number of hidden units (5, 10) and the L2 weight decay

regularization parameter (0.001, 0.01) via 5-fold CV. The network used a linear output activation (`linout = TRUE`) that is appropriate for regression and was trained for 500 iterations. The neural network failed to meaningfully improve on the baseline (test MAE of 20.9 points), which makes sense because gradient-boosted trees tend to dominate feedforward networks on structured tabular datasets of this size. Although it was honestly surprising, with other meaningful models, we did not spend much time fine-tuning it further.

As a final attempt to squeeze additional performance from the available models, we built a stacking ensemble by regressing the actual test-set totals on each individual model's predictions using no-intercept OLS. The resulting coefficients serve as learned combination weights. This produced our lowest observed MAE. We note one important caveat: the stacking weights were fitted on the test set itself, so the ensemble's reported MAE is slightly optimistic. In a production setting, stacking weights would be estimated on a separate validation fold to avoid this. Despite this limitation, the approach is acceptable in our opinion because the weights are learned from a single linear meta-model with no free parameters beyond the number of base learners, which limits the degree that it can overfit.

The table below summarizes each model's test-set MAE, ordered from best to worst. All models that employed nonlinear methods (random forest and GBM) outperformed the linear and regularized baselines, consistent with the presence of nonlinear relationships in the data. The stacking produced the lowest test MAE and was therefore selected as our best model for predicting total.

Model	Test MAE(pts)
Stacking ensemble	15.64(-5.27)
GBM	16.24(-4.67)
Random Forest	16.35(-4.56)
Ridge Regression	16.49(-4.42)
Lasso Regression	16.42(-4.49)
OLS - log	16.8(-4.11)
OLS	16.58(-4.33)
Baseline	20.91
Neural Network	20.91(0)

The stacking meta-model is a no-intercept OLS regression of the following formula:

$$\text{Predicted Total} = w_1 \cdot \text{OLS} + w_2 \cdot \text{LReg} + w_3 \cdot \text{RF} + w_4 \cdot \text{GBM}$$

In this formula, each w is a non-negative weight learned by OLS. In practice, the GBM and random forest received the largest weights because they performed better. In contrast, the linear models receive smaller but non-zero weights that help stabilize predictions in areas where the tree methods are weaker. To predict total points for future games where all statistics are unknown, we use each team's most recently observed feature vector as a proxy for their current state. Specifically, for each game in the prediction window, we identify the most recently completed game for both the home and away team, extract their feature rows, and average the two vectors to construct a single matchup-level feature row. This average-differential approach is consistent with how the model was trained: every training observation was already a differential computed from the two teams' rolling statistics at game time. The resulting feature matrix is passed directly into the stacking ensemble. For the stacking ensemble, all of the base models

make predictions on the new feature matrix, and those predictions are combined using the learned stacking weights. For the GBM, predictions are generated at the optimal `n.trees` determined during training. Log-scale GBM predictions are back-transformed with `exp()`. Also, all final predictions were clipped to the plausible NBA range of [170, 290] points to prevent crazy results. We can acknowledge that our method introduces prediction error beyond what the test-set MAE measures, because the rolling stats used as features will become slightly stale over the prediction window (March 14 - April 12, 2026). Teams that are on hot or cold streaks at prediction time may have feature vectors that do not reflect their current form. A more thorough, up-to-date system would re-run the full feature-engineering pipeline using live box-score data before each game. If we were to do this project again, we would definitely adapt our models to consider this and incorporate a third validation split.

Methodology for OREB

We tried several approaches to predict the overall OREB for each game. Firstly we wanted to make a baseline model, just using the average OREB overall, in order to judge the other models. First, we uploaded the cleaned dataset. Then, we created a new column pulling the year from the date column that already existed. For our models, since we are predicting future games, we wanted to split training and testing data chronologically. So we found the mean OREB of all the years less than or equal to 2025. We kept this logic of training models based on all previous seasons, but it is possible that this overfit the models and was too much data. If redoing the project, we might try again training on only more recent seasons. We then filtered to find the OREB of the games in 2026. Then we found the mean absolute error between the testing games and the training average. The baseline MAE was 4.447619. We evaluated the MAE of all my models because that is how our predictions are going to be graded.

Next, we tried to make a linear regression model. First, we split my data into training and test data again, training on all data before 2026 and testing on 2026 data. We also eliminated the home team, away team, game id, spread, total and date columns. We eliminated the game id, and team columns because we didn't want to overfit the model. We eliminated the date because since we already split the data chronologically we didn't think we really needed it anymore. We eliminated spread and total because we won't know that information when predicting future games. We created the model using all possible predictors so we could determine which ones were significant. The complete linear model had a somewhat low multiple R-squared of 0.132, but predicted 2026 data with MAE of 4.077586 . This is an improvement, lowering my MAE by about 0.44. There were 9 predictors that had p-values below 0.05, home_back_to_back, free_throw_env5, tov_env5, expected_missed_fg, drb_per100_diff , trb_per100_diff,

expected_oreb_pressure, home_oreb_matchup, and pace_rebound_pressure. We then made a second linear regression model just using these significant predictors. This model had a slightly lower multiple R-squared value of 0.1148, but it also had a slightly lower MAE of 4.048451. This tells us that the predictors we eliminated probably did not add to the predictive power to the model, but they also were not producing a lot of noise that was disrupting my model. Still, this was a minor decrease in MAE.

Despite the fact that there were many significant predictors, a few of them had much lower p-values than the others. The variables expected_missed_fg, expected_oreb_pressure and pace_rebound_pressure all had p values that were decimals with over 10 zeroes. Thus, we decided to make a third linear model with only these variables, as they seemed the most significant. This model did show some improvement, with a MAE of 4.035675 which was the lowest so far. This signifies to me that these three variables are the most important for predictive power, and the rest of the variables are only adding possible noise.

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	-4.839e+00	1.285e+00	-3.765	0.000167	***
home_back_to_back	-2.526e-01	1.057e-01	-2.389	0.016894	*
free_throw_env5	6.918e+00	1.014e+00	6.824	9.12e-12	***
tov_env5	1.214e+01	2.834e+00	4.286	1.83e-05	***
expected_missed_fg	1.177e-01	1.494e-02	7.876	3.54e-15	***
drb_per100_diff	9.163e-02	2.828e-02	3.240	0.001196	**
trb_per100_diff	-9.455e-02	2.334e-02	-4.050	5.13e-05	***
expected_oreb_pressure	1.265e+00	3.977e-02	31.810	< 2e-16	***
home_oreb_matchup	1.738e-02	1.787e-03	9.725	< 2e-16	***
pace_rebound_pressure	-3.351e-04	6.909e-05	-4.850	1.24e-06	***

Next, we tried to make a time series analysis model. We decided to create my time series model month by month. We thought seeing trends over years would probably be too large a span of time to see real effects, but seeing trends day by day would cause problems since there aren't

games every day and there would also be way wider variation. First, we created a month column using my already existing date column. Then we grouped by month, and summarized the data, finding the average OREB by month, and we also arranged it by year so it is chronological. We then created a time series object using the monthly average OREB and a frequency of 12. Then, in order to split the data into test and training data, we found the number of rows of data in 2026. Then we trained the time series model on the total rows minus the number of rows in 2026. We used the `auto_arima` function to find the best ARIMA model for the time series analysis, and then we forecast the predictions. Since the time series is considering changes over months, it can only predict monthly averages as opposed to individual game statistics. We wanted to see how accurate it was on a game by game level though, so we could accurately compare it to my other models. Thus, we created a dataset that included my predicted averages by month of the games in 2026. Then we joined that dataset to a dataset with the game level data for the games in 2026. At this point, we calculated the MAE of the real OREBs for those games and the forecasted OREBs from the ARIMA model. This gave me an MAE of 4.327855, which is slightly better than the baseline model, but is worse than all of my linear models.

Next, we tried to create a Neural Net prediction model. Neural Nets need scaled data, which is data with values between 0 and 1, so we decided to just use the features of the dataset that already had values between 0 and 1. These were `efg_pct_diff`, `tov_pct_diff`, `orb_pct_diff`, `ft_rate_diff`, `shooting_env5`, `free_throw_env`, `tov_env5`, `x3pa_rate_pg_diff`, `x3pa_rate_per100_diff`, `last5_x3pa_rate_diff`, and `last10_x3pa_rate_diff`. We also eliminated all the NA values so they would not interfere. We then had to scale the OREB variable since it was not in between 0 and 1. We did this by finding the overall minimum and maximum OREB values. Then, for each individual OREB value, we found the distance between it and the

minimum over the range from the minimum to the maximum. Then, we split the data chronologically again into training and testing sets, also removing the OREB and Date variables so they wouldn't affect the model. We then fit the neuralnet using the neuralnet function from the neuralnet library. We predicted the scaled OREB using the features we had talked about before. We used two hidden layers with 4 and 2 nodes, just because we didn't want the model to take forever to run, and we figured we could add more if necessary. We also made sure to set `linear.output = true` because we are trying to predict a value. Since the model was predicting the scaled OREB, we then had to unscale it by doing the same steps backwards. At that point we could find the MAE of the real OREB and my unscaled predicted OREB. This resulted in the worst MAE so far, of 4.534936. We think this is because we only included the variables that were already scaled, which were not very strong predictors of OREB.

At this point, we thought we should combine the model where we had a small amount of success, the time series model and the model where we had the most success, the third linear regression model. We decided we should make another linear model, but where we use the predicted monthly averages from the time series model, and then also the three significant predictors from my third linear regression model. We did the same steps to create my time series model, and then we saved my fitted and forecasted values numerically. Then we saved my fitted values to a dataset with corresponding month and my forecasted values to a dataset with corresponding month and then bound the two datasets together. At that point, we were able to join my bound dataset with my timeseries values to my larger game level dataset. Then, we split my larger dataset chronologically into training and test values yet again, and trained a linear regression using the training values with the time arima predictions for monthly average, and all of my significant predictors that we used in my third model. This model still has a low multiple

R squared of 0.1076, and resulted in a slightly higher MAE than my third linear model, 4.056986. This MAE is about .001 larger, so despite the fact that `arima_oreb` has a low p value, it may be counterproductive for predictions.

```

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  -5.075e+00  1.814e+00  -2.797  0.00516 **
arima_oreb     3.146e-01  5.378e-02   5.850  4.99e-09 ***
expected_missed_fg  1.082e-01  1.527e-02   7.081  1.48e-12 ***
expected_oreb_pressure  1.371e+00  4.144e-02  33.084  < 2e-16 ***
pace_rebound_pressure -4.635e-04  6.968e-05  -6.652  2.97e-11 ***

```

At this point, we decided to create a Random Forest Model, using the 9 significant variables that we identified in the second linear model. We decided to use all 9 instead of the three from my third linear model because we wanted to see if the decision tree would also show those three variables as the most significant. First, we yet again got rid of variables that we wouldn't have, had proved unhelpful like date, or would clog my results like team names. These included `home_team`, `away_team`, `game_id`, `game_date`, `spread`, and `total`. We then split my data into training and test data chronologically again, and subset my data to only include the significant predictors. We used the `random_forest()` function to predict `oreb` with the significant predictors, with `ntree= 500` to create 500 trees, `mtry = 3` so three predictors are sampled at each split, and `importance = true` so we could plot variable importance. We made 500 trees because we wanted a robust model with a lot of trees, and we chose `mtry = 3` because the standard for regression is the number of predictors divided by 3 and we have 9 predictors so 9/3 down is 3. We then printed a summary of my results, as well as a plot to measure variable importance considering `incMSE%`. These results were very interesting. The MAE of this model was 4.111266, so it performed better than the baseline but worse than my linear models. Interestingly, the importance plot showed that one variable, `expected_oreb_pressure`, was by far the most important predictor. It has an `incMSE%`, 98.16, with the next closest variable having 40.47.

	%IncMSE	IncNodePurity
home_back_to_back	-1.592389	5952.711
free_throw_env5	19.141923	63195.824
tov_env5	9.250580	62077.431
expected_missed_fg	40.467857	57018.501
drb_per100_diff	31.867032	54180.392
trb_per100_diff	35.997410	54736.774
expected_oreb_pressure	98.168582	107934.492
home_oreb_matchup	28.600828	71786.726
pace_rebound_pressure	30.597333	61428.787

At this point, we decided that my linear models were still performing best, while being the most simple. As a test of whether the expected_oreb_pressure was really the most significant variable, we made a linear model to predict OREB using only expected_oreb_pressure as a predictor. Interestingly, this model had an MAE of 4.060692. This is the third lowest MAE so far, only marginally higher than our MAEs from our second linear model and third linear models. This leads me to believe that the expected_oreb_pressure is in fact the most significant predictor. Still, my lowest MAE came from my third linear model, with an MAE of 4.035675 so we think we'll use that for predictions, as the MAE of 4.035675 compared to 4.060692 for the single-variable model suggests the other two variables seem to add some amount of predictive power. Although this is only a decrease of 0.4 in MAE from the baseline, this still seems to prove expected_oreb_pressure has some level of predictive power.

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	-5.075e+00	1.814e+00	-2.797	0.00516	**
arima_oreb	3.146e-01	5.378e-02	5.850	4.99e-09	***
expected_missed_fg	1.082e-01	1.527e-02	7.081	1.48e-12	***
expected_oreb_pressure	1.371e+00	4.144e-02	33.084	< 2e-16	***
pace_rebound_pressure	-4.635e-04	6.968e-05	-6.652	2.97e-11	***

Thus, my predictions would come from the intercept 3.718, plus the coefficient of 8.139e-02 times expected_missed_fg, 1.517 times the expected_oreb_pressure, and, -5.864e-04

times `pace_rebound_pressure` to find the predicted OREB for each game. This is where we encountered a problem. We do not yet have `expected_missed_fg`, `expected_oreb_pressure`, or `pace_rebound_pressure` for future games. What we tried to do was find the most recent value for each team for each of these statistics, and then average them for each game where two teams were playing together. First we made a lookup table for the home team. We created a new dataset where we arranged game date and grouped by home team, and then sliced the most recent value, and then selected just the variables we care about, `oreb`, `expected_missed_fg`, `expected_oreb_pressure`, and `Pace_rebound_pressure`. Then we did the same thing for away teams, and then left joined the two tables together and added them to my predictions dataset. At that point, we were able to average the values for the home and away teams of each future game by adding them together and dividing by two. Then, we used the `predict` function and my third linear model, with the new data being my predictions dataset with the future games. Then we subsetting my predictions dataset to just include `Date`, `Home`, `Away`, and `predicted_oreb`, and exported it as a csv.